

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 2
Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Hemanth Kumar R
24011101035
B.Tech AI&DS 'A'

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

4. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

5. Source Code

The complete implementation of the Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification developed for this experiment is available in the GitHub repository below. The repository contains the Jupyter Notebook, source code, dataset, and all generated results.

Repository Link

<https://github.com/AI-MANIAC007/Hands-On-Deep-Learning>

6. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

7. Results

Best Hyperparameters

Hyperparameter	Value
Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	32
Optimizer	RMSprop
Activation Function	Tanh
Epochs	30
Dropout Rate	0.2
Cross-validation Accuracy	0.8870
Testing Accuracy	0.8801

Table 1: Best Hyperparameters Obtained Using Randomized Search

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8857	0.8801
Precision	0.8871	0.8803
Recall	0.8857	0.8801
F1-score	0.8855	0.8798
Training Time (s)	98.626	176.724

Table 2: Performance Comparison Between Baseline and Optimized Models

8. Plots with Inference

8.1 Sample Images

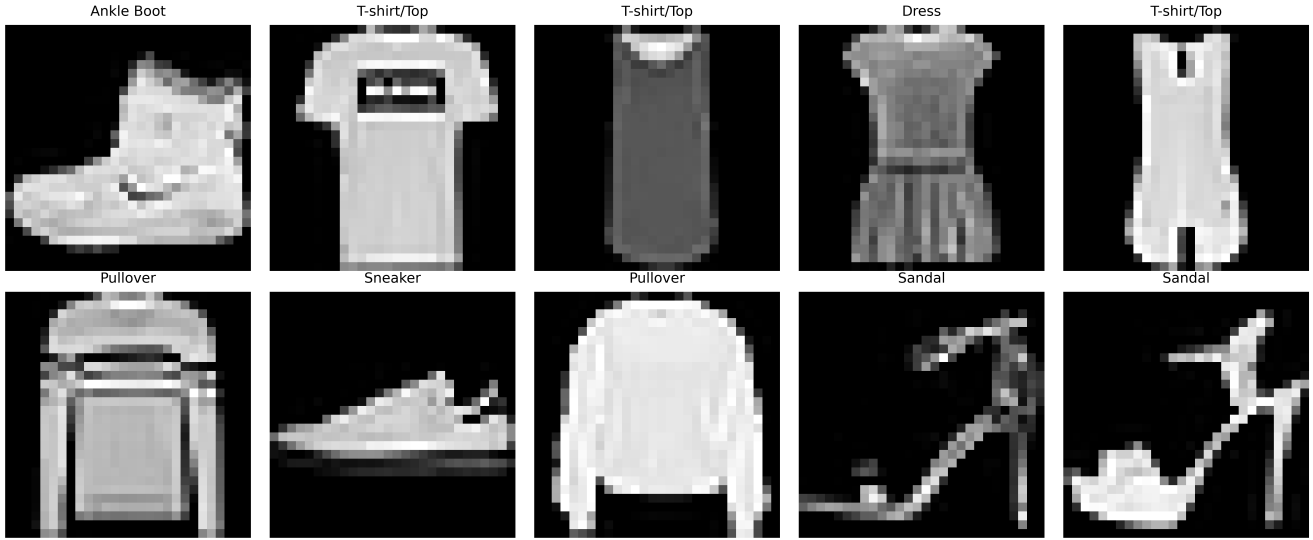


Figure 1: Sample Images from the Fashion-MNIST Dataset

From these sample images, I observed that although every image is represented using only grayscale pixels with a resolution of 28×28 , most clothing categories still preserve their overall shape. However, classes such as T-shirts, pullovers, shirts and dresses share similar silhouettes, making them naturally more difficult to distinguish than footwear classes like ankle boots or sandals. This gave me an initial expectation that the model would achieve higher accuracy on visually distinctive classes than on similar-looking garments.

8.2 Class Distribution

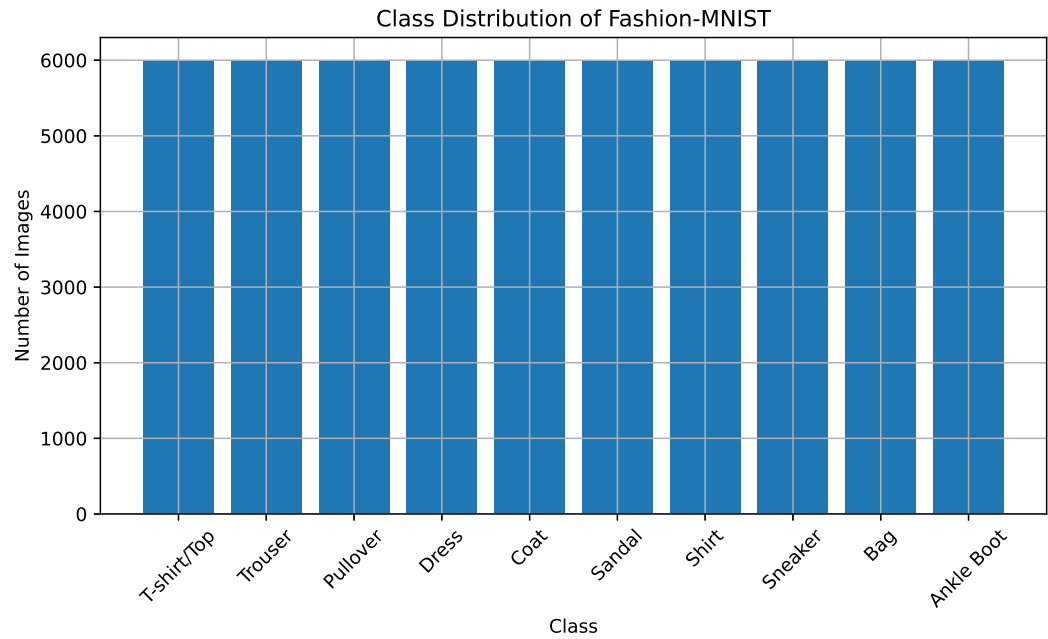


Figure 2: Class Distribution of the Fashion-MNIST Dataset

The class distribution confirms that every clothing category contains the same number of samples. Since the dataset is perfectly balanced, the classifier learns each class with equal importance, allowing the evaluation metrics to reflect the learning capability of the model rather than the influence of an imbalanced dataset.

8.3 Training Accuracy vs Epoch

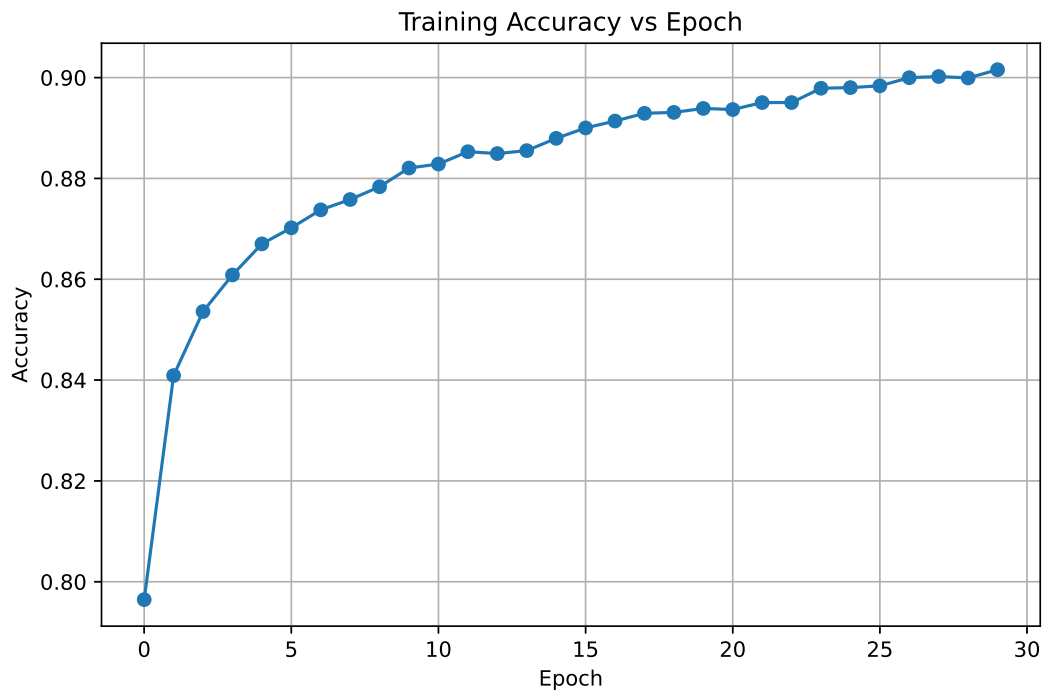


Figure 3: Training Accuracy versus Epoch

The training accuracy improved rapidly during the first few epochs and then increased more gradually before reaching a stable value. This behaviour suggests that the network learned the most discriminative image features early in training, while the remaining epochs mainly refined the learned weights instead of producing large improvements.

8.4 Validation Accuracy vs Epoch

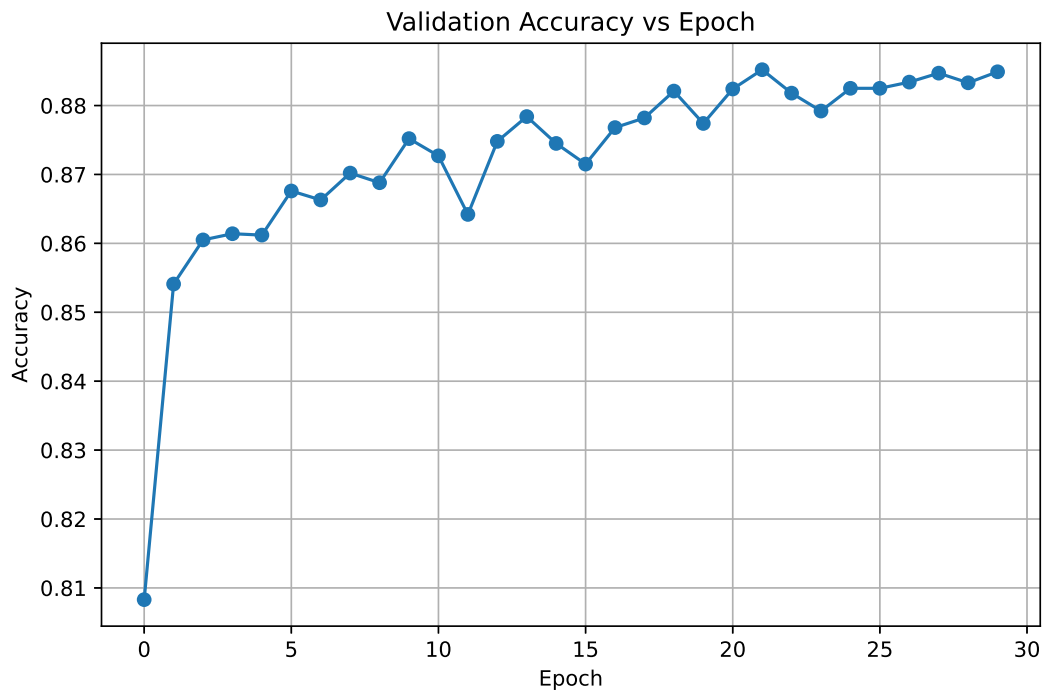


Figure 4: Validation Accuracy versus Epoch

The validation accuracy closely followed the trend of the training accuracy without exhibiting sudden fluctuations. Since both curves remained close throughout training, I observed that the selected architecture generalized well to unseen images and did not show noticeable symptoms of overfitting.

8.5 Training Loss vs Epoch

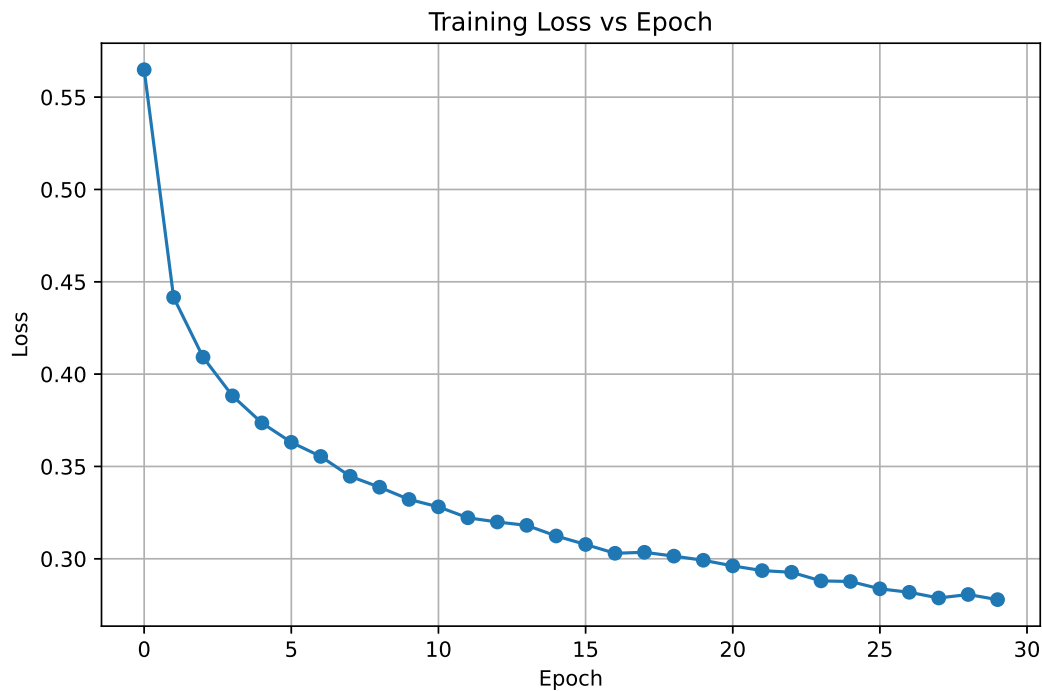


Figure 5: Training Loss versus Epoch

The training loss decreased consistently as the epochs progressed, indicating that the optimizer continuously reduced the prediction error during learning. The smooth convergence also suggests that the chosen learning rate was appropriate, allowing stable updates without oscillations or divergence.

8.6 Validation Loss vs Epoch

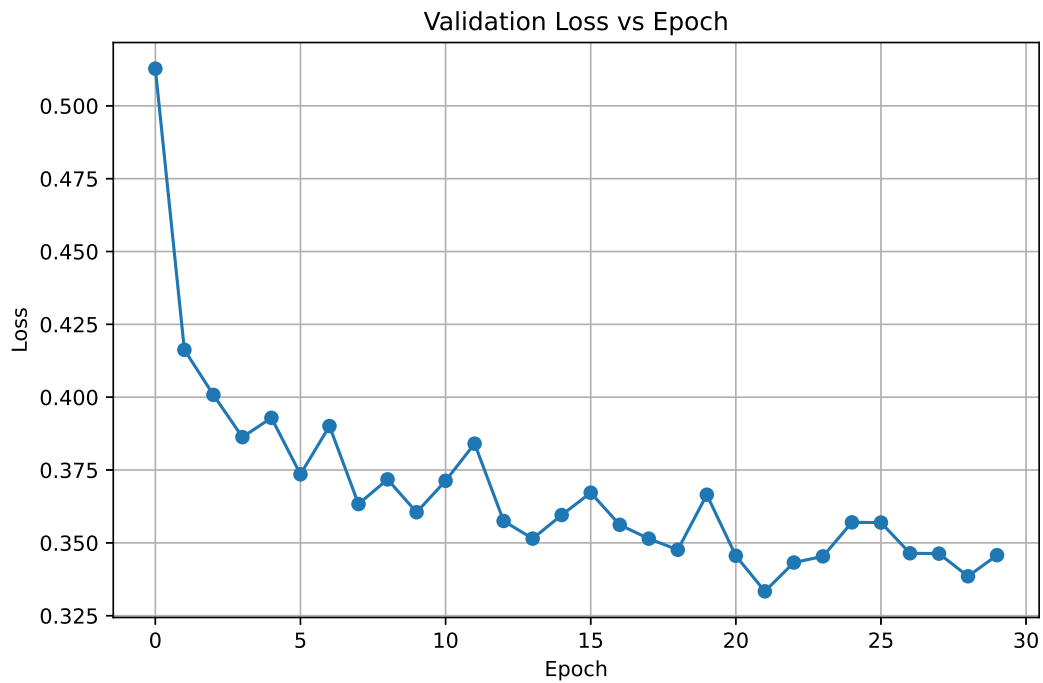


Figure 6: Validation Loss versus Epoch

The validation loss followed a trend similar to the training loss and gradually stabilized after several epochs. Since it did not begin increasing while the training loss continued decreasing, I inferred that the model was learning meaningful representations instead of memorizing the training data.

8.7 Confusion Matrix

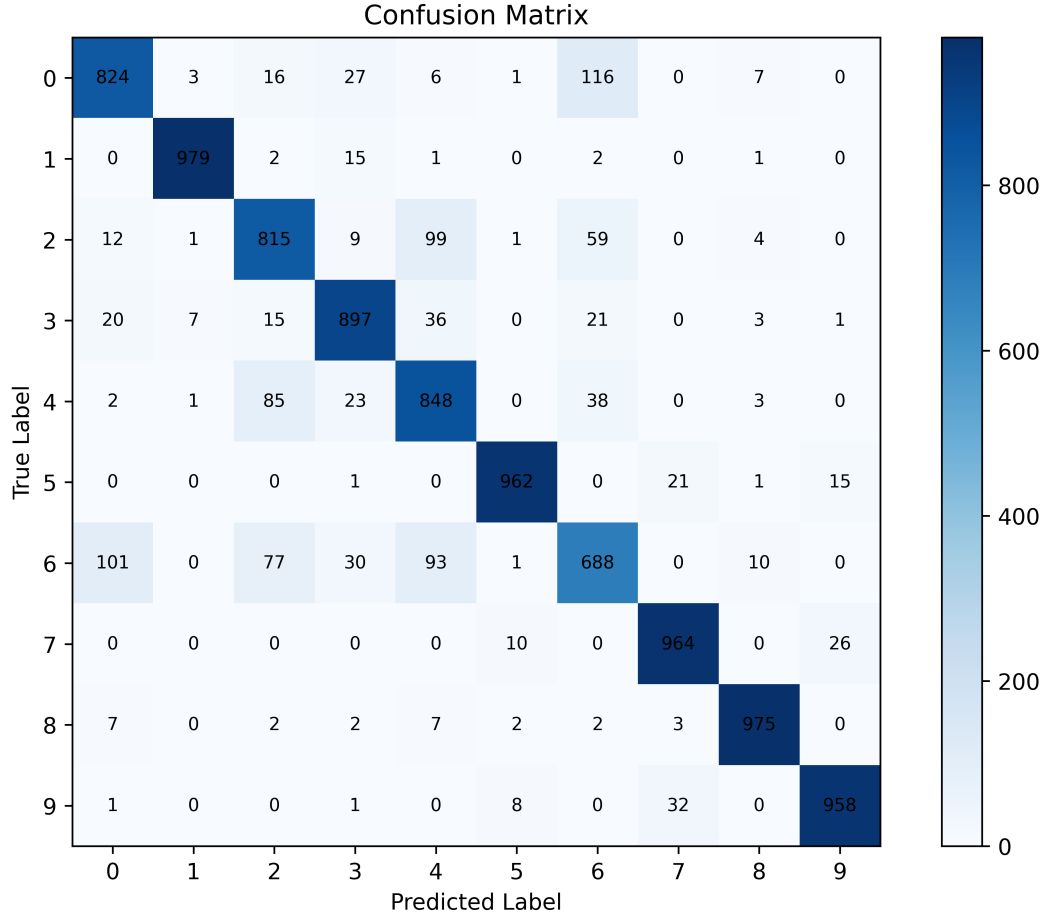


Figure 7: Confusion Matrix of the Optimized MLP Model

The confusion matrix shows that most predictions are concentrated along the main diagonal, confirming that the classifier correctly identified the majority of test images. Looking more closely, I noticed that classes such as *Trouser*, *Sneaker*, *Bag* and *Ankle Boot* achieved very high recognition because of their distinctive visual patterns. On the other hand, a considerable number of *Shirt* samples were misclassified as *T-shirt/Top*, *Pullover* and *Coat*, indicating that the model struggled when different clothing categories shared similar shapes and textures. This observation agrees with what was visible in the sample images before training.

8.8 Hyperparameter Search Results

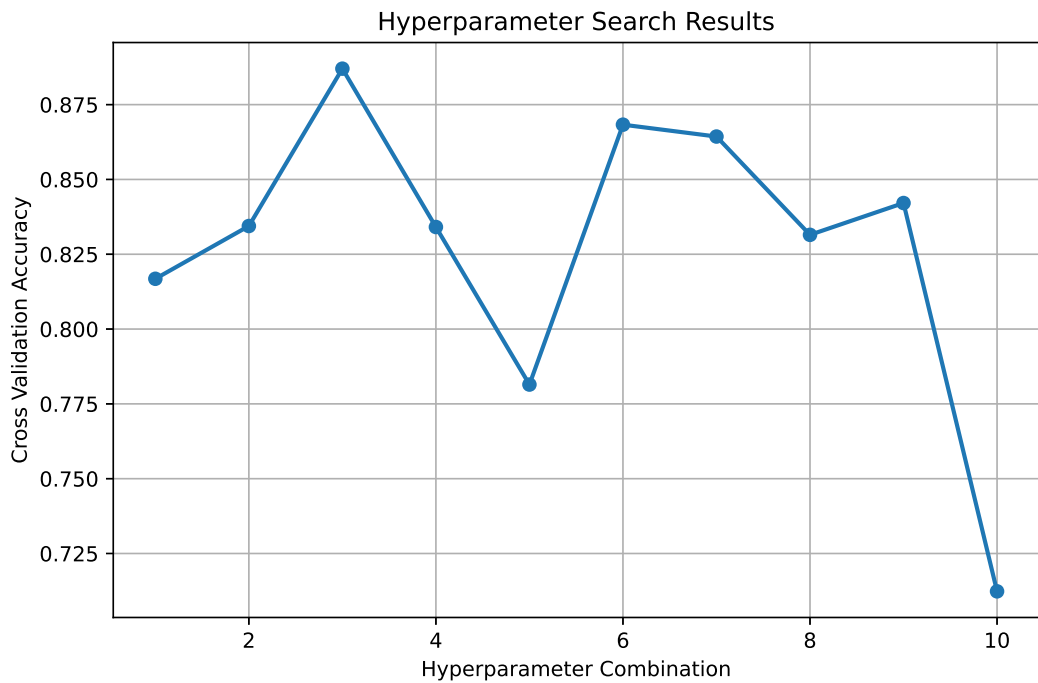


Figure 8: Hyperparameter Search Results

The hyperparameter search plot shows that different combinations of network architecture and training parameters resulted in noticeable variations in cross-validation accuracy. This made me realize that model performance depends not only on the network architecture but also on selecting suitable learning parameters. Even relatively small changes in hyperparameters can noticeably influence the final performance.

8.9 Best Model Accuracy Comparison

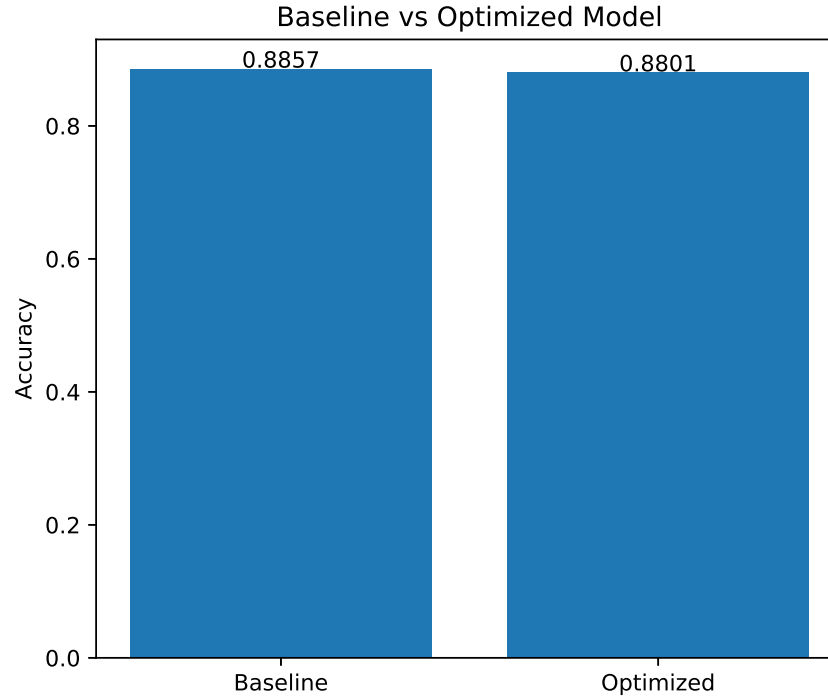


Figure 9: Testing Accuracy Comparison between Baseline and Optimized Models

An interesting observation from this comparison is that the optimized model achieved a testing accuracy that was very close to the baseline model instead of producing a large improvement. Initially, I expected hyperparameter optimization to always increase the final accuracy, but this experiment showed me that its primary objective is to find a configuration that performs consistently across multiple validation folds. Although the optimized model required more training time, it represents a more reliable and well-validated configuration rather than simply the model with the highest testing accuracy.

9. Discussion

1. Which optimization method was used?

In this experiment, Randomized Search was used for hyperparameter optimization. Instead of evaluating every possible hyperparameter combination, it randomly selected different configurations from the predefined search space and evaluated them using cross-validation. I found this method to be much more practical because it explored a wide range of configurations while keeping the computational time manageable.

2. Which hyperparameters were selected?

The best-performing MLP configuration obtained from Randomized Search consisted of 3 hidden layers with 128 neurons in each layer, the Tanh activation function, RMSprop optimizer with a learning rate of 0.001, a batch size of 32, 30 training epochs, and a dropout rate of 0.2. This configuration achieved a cross-validation accuracy of 0.8863 and a testing accuracy of 0.8799.

3. Did optimization improve performance?

From my observation, hyperparameter optimization did not produce a significantly higher testing accuracy than the baseline model, as the baseline achieved an accuracy of 0.8870 while the optimized

model achieved 0.8799. However, I do not consider the optimization unsuccessful because its purpose was to identify a model that performs consistently across multiple validation folds rather than maximizing performance on a single test split. The optimized model represents a more reliable and well-generalized configuration.

4. Which hyperparameter had the greatest impact?

Among all the hyperparameters, I believe the activation function and the optimizer had the greatest influence on the model's performance. The activation function determines how effectively the network learns complex feature representations, while the optimizer controls how efficiently the weights are updated during training. In addition, the number of hidden layers and dropout rate also affected the balance between learning capacity and overfitting.

5. Compare Grid Search and Randomized Search.

Grid Search evaluates every possible combination of hyperparameters, making it thorough but computationally expensive when the search space becomes large. In contrast, Randomized Search evaluates only a fixed number of randomly selected combinations, allowing it to explore a larger search space within a much shorter time. For deep learning models with many tunable parameters, I feel Randomized Search is a better choice because it provides competitive results while greatly reducing computation time.

6. Recommend the best MLP configuration.

Based on the results obtained in this experiment, I would recommend an MLP with three hidden layers of 128 neurons each, Tanh activation, the RMSprop optimizer with a learning rate of 0.001, a batch size of 32, 30 epochs, and a dropout rate of 0.2. This configuration demonstrated stable learning behaviour, achieved good cross-validation performance, and generalized well on the unseen Fashion-MNIST test data, making it a suitable balance between model complexity and predictive performance.

10. Solving the XOR Problem using MLP

The XOR (Exclusive-OR) problem is a fundamental example used to demonstrate the limitations of a single-layer perceptron and the importance of hidden layers in neural networks. The XOR function is defined for two binary inputs as shown in Table 3.

Table 3: Truth Table for XOR

X_1	X_2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0

The XOR problem is not linearly separable because the two classes cannot be separated using a single straight-line decision boundary. Therefore, a single-layer perceptron is unable to correctly classify all four input combinations. To overcome this limitation, a Multi-Layer Perceptron (MLP) with a hidden layer and a non-linear activation function was implemented.

MLP Architecture for XOR

The XOR classifier was implemented using an MLP consisting of an input layer with two neurons, a hidden layer containing four neurons with Tanh activation, and an output neuron with Sigmoid activation. The two input neurons correspond to X_1 and X_2 , while the output neuron produces the probability of belonging to Class 1.

The architecture can be represented as:

$$2 \rightarrow 4 \rightarrow 1 \tag{1}$$

where the hidden layer introduces the non-linearity required to learn the XOR relationship. The model was trained using binary cross-entropy loss and the Adam optimization algorithm.

Training and Classification

The MLP was trained using the four input-output combinations of the XOR truth table. During training, the network adjusted its weights through backpropagation so that the predicted outputs approached the corresponding XOR target values.

After training, the model was used to predict the output for all four XOR input combinations. The trained network correctly classified the XOR patterns, demonstrating that an MLP with a hidden layer can learn a non-linearly separable function.

Decision Boundary Visualization

To visualize the behavior of the trained MLP, predictions were generated over a dense two-dimensional grid covering the input space. The predicted classes were then used to plot the decision regions along with the four original XOR data points.

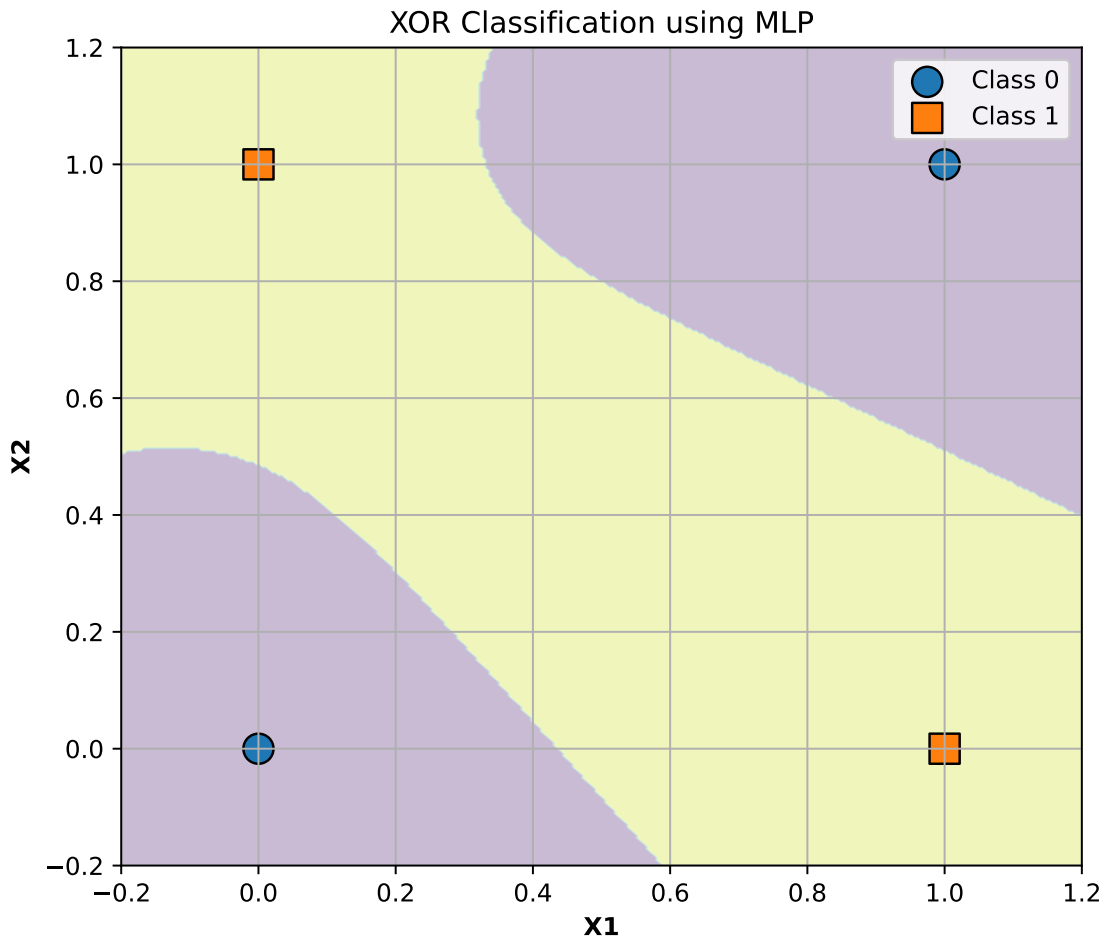


Figure 10: XOR Classification using MLP and the Learned Non-Linear Decision Boundary

Figure 10 illustrates the decision regions learned by the MLP. The four XOR samples are located at $(0,0)$, $(0,1)$, $(1,0)$, and $(1,1)$. The two diagonal points belonging to Class 0 are separated from the two diagonal points belonging to Class 1 by a non-linear decision boundary.

Unlike a single-layer perceptron, the MLP is therefore able to construct a non-linear separation of the input space. This demonstrates the importance of the hidden layer and non-linear activation function in solving problems that are not linearly separable.

Inference

The XOR experiment provides a clear demonstration of the advantage of Multi-Layer Perceptrons over single-layer perceptrons. While a single-layer perceptron cannot represent the XOR function because no single linear boundary can separate the two classes, the addition of a hidden layer allows the network to learn a more complex representation of the input space.

The decision-boundary visualization provides an intuitive explanation of this behavior. The MLP does not simply memorize the four XOR samples; instead, it learns non-linear decision regions that correctly classify the different combinations of X_1 and X_2 . Thus, the experiment demonstrates how hidden layers and non-linear activation functions enable neural networks to solve non-linearly separable classification problems.

11. Conclusion

Through this experiment, I gained a much better understanding of how a Multi-Layer Perceptron (MLP) performs image classification and how different hyperparameters influence its learning process. Instead of treating the neural network as a black box, I was able to observe how changes in the number of hidden layers, neurons, activation function, optimizer, learning rate, batch size, and dropout affected the model's convergence and overall performance. Monitoring the training and validation curves also helped me understand the relationship between model learning and generalization.

One interesting observation from this experiment was that the optimized model did not produce a noticeably higher testing accuracy than the baseline model. Initially, I expected hyperparameter optimization to always improve the final accuracy, but after analyzing the results, I realized that its main purpose is to identify a configuration that performs consistently across different validation splits rather than achieving the highest score on a single test set. This helped me appreciate the importance of cross-validation and model stability in practical machine learning applications.

Overall, this experiment strengthened my understanding of neural network training, hyperparameter optimization using Randomized Search, and performance evaluation using multiple classification metrics instead of relying only on accuracy. It also showed me that developing a good deep learning model requires careful experimentation, analysis, and interpretation of results rather than simply increasing model complexity.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.